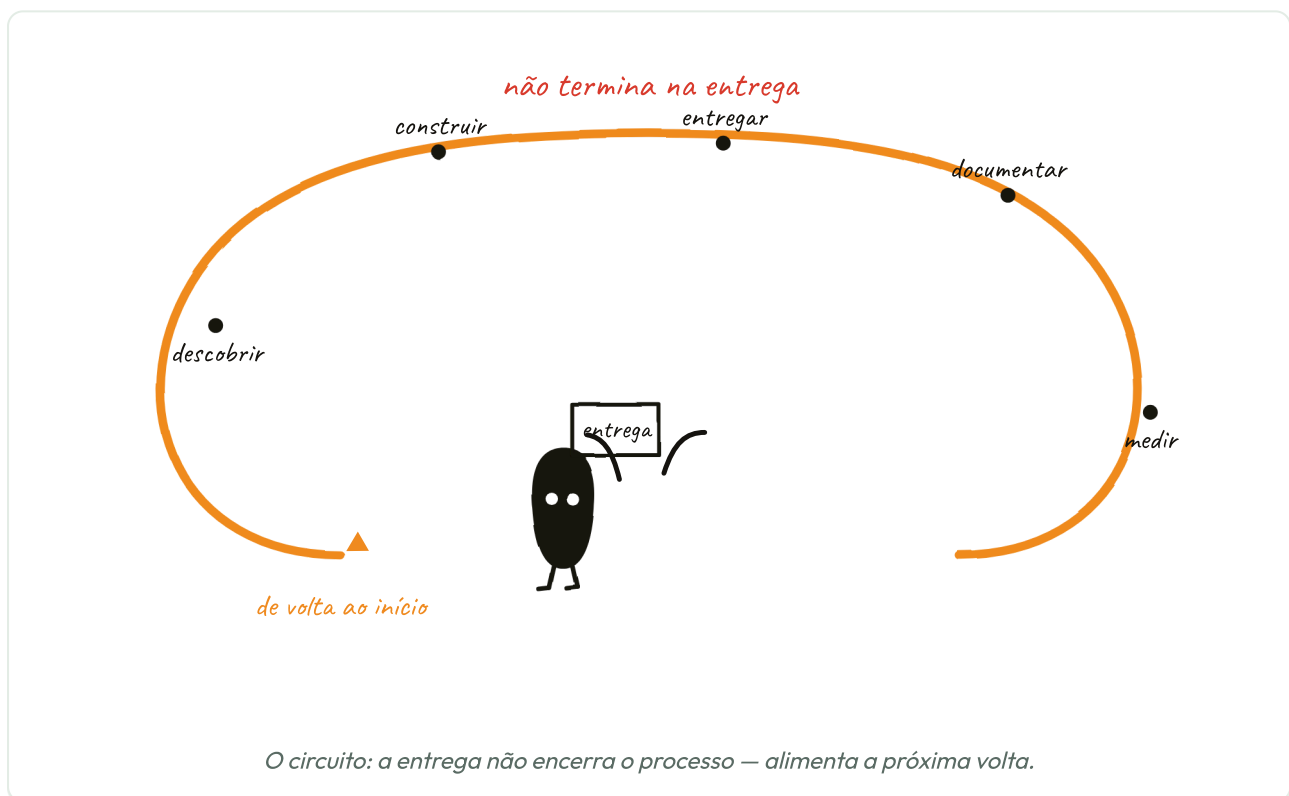


DOCUMENTO TÉCNICO INTERNO · V2.0

Processo de UX Ponta a Ponta

Code-First, Documentação Viva e Melhoria Contínua

O código permanece como fonte única da verdade — e o ciclo passa a se fechar com documentação automatizada (Supernova + Figma) e medição contínua de uso (PostHog). UX deixa de terminar na entrega: vira um circuito.



Autoria	Arthur Meireles — UX, GB Agritech
Base técnica	Projeto CERNE (React + TypeScript + Chakra UI 3 + tokens) · Supernova · PostHog
Versão	2.0 — Junho/2026 · substitui a v1.0 (Processo de UX Code-First)

Sumário Executivo

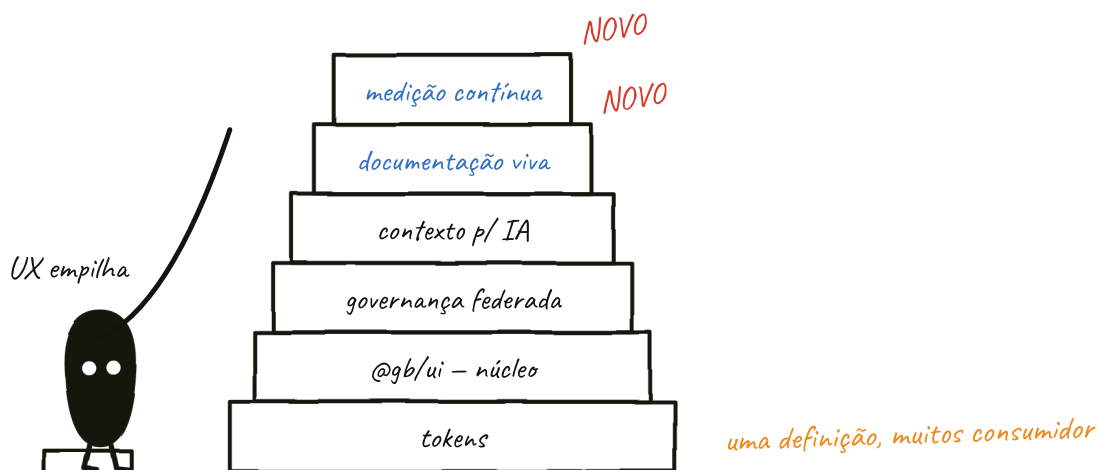
Onde estamos × para onde vamos. Em produção hoje: tokens centralizados em código como fonte única (`src/design/tokens.ts`), catálogo de componentes em `src/components/ui/` , Storybook, Tailwind e o contexto para IA (Leis e Regras). Alvo desta v2 (a construir): exportar os tokens no padrão W3C DTCG, reconstruir o núcleo sobre Chakra UI 3 com recipes, distribuir os pacotes `@gb/tokens` / `@gb/ui` / `@gb/lint` , documentar no Supernova e instrumentar com PostHog. Os marcos descritos adiante são o **destino arquitetural**, não o estado atual.

A v1.0 deste documento consolidou o processo *code-first*: os tokens como fonte única em código, um catálogo de componentes curados, Storybook e o contexto para IA (as Leis do CERNE). Esse alicerce está em produção. A **formalização** desse núcleo — tokens no padrão W3C DTCG, componentes reconstruídos sobre Chakra UI 3 e distribuídos como pacotes `@gb/*` versionados — é a **arquitetura-alvo** desta v2 (Seções 7 e 8), validada pela pesquisa de indústria como o padrão consolidado de 2025–2026.

Esta v2.0 estende o processo nas duas pontas que a v1 deixava em aberto: **o que acontece depois da entrega**. A resposta tem três movimentos:

Extensão	O que muda	Ferramenta
Claude no time de UX	O Claude deixa de ser ferramenta só dos devs: o próprio time de UX o usa para produzir tokens, componentes, telas, stories e auditorias — sob as Leis do CERNE.	Claude Code + skill <code>gb-design-system</code>
Documentação como subproduto	Após a entrega, a documentação é montada no Supernova e no Figma puxando os tokens automaticamente do código — documentar deixa de onerar o time.	Supernova (free) + Tokens Studio + Figma
Melhoria contínua	Telemetria de uso real (replays, funis, surveys, feature flags) fecha o ciclo: o que o usuário faz na tela vira insumo do próximo sprint de UX.	PostHog (free)

Os seis pilares do processo



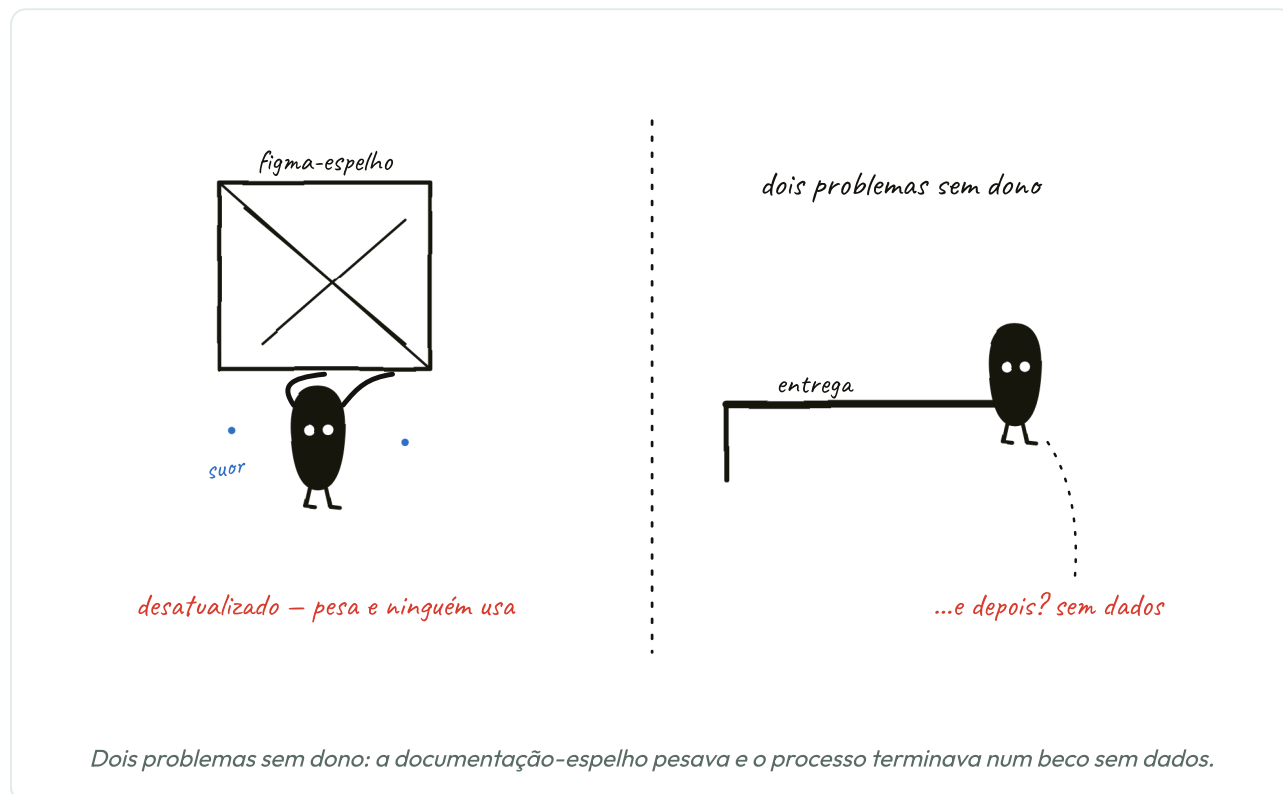
Camada sobre camada: os quatro pilares da v1 sustentam os dois novos — documentar e medir.

- ◆ **Tokens como constituição** — valores de design vivem em código como fonte única (hoje em `tokens.ts`); **ALVO** versioná-los no padrão W3C DTCG para servir todos os produtos da GB.
- ◆ **Núcleo de componentes** `@gb/ui` **ALVO** — primitivas curadas a serem reconstruídas sobre Chakra UI 3 (recipes e slot recipes) e distribuídas como pacote npm versionado: uma correção no núcleo propaga para todos os consumidores. Hoje as primitivas vivem em `src/components/ui/`.
- ◆ **Governança federada** — UX é a fonte central dos tokens; devs criam componentes livremente, e os melhores são promovidos ao núcleo com aprovação de UX.
- ◆ **Contexto para IA** — Leis e receitas de composição empacotadas como contexto legível por agentes, permitindo construir telas novas com consistência mesmo sem Figma.
- ◆ **NOVO Documentação viva** — Supernova conectado ao código e ao Figma; tokens fluem automaticamente, documentação nasce depois da entrega, sem retrabalho manual.
- ◆ **NOVO Medição contínua** — PostHog instrumenta o produto; dados de comportamento real alimentam hipóteses, experimentos e o backlog de UX.

Custo de entrada: R\$ 0. Todo o ferramental novo desta versão opera em planos gratuitos no primeiro momento — Supernova Free (até 5 usuários, 1 design system, 20 páginas de documentação) e PostHog Free (1M eventos, 5 mil session replays e 1.500 respostas de survey por mês). Upgrade só quando a fricção justificar, com critérios definidos na Seção 9.

Contexto e o Que Faltava

O fluxo tradicional — desenhar no Figma e repassar para implementação — foi invertido no CERNE: o código nasce com alta fidelidade e governança (as Leis do projeto), e o Figma deixou de ser entregável obrigatório. A v1 resolveu a *produção*: como construir telas consistentes em escala, com IA, sem mockup.



Dois problemas, porém, permaneciam sem dono:

1 — A documentação onerava o time

Converter o que existe em código para o Figma era um passo manual, custoso e que quebrava os vínculos com o design system. O resultado: um Figma-espelho sempre desatualizado, com percepção (correta) de baixo valor. A consequência prática era documentação adiada ou abandonada — e o conhecimento do sistema concentrado em quem o construiu (*bus-factor*).

2 — O processo terminava na entrega

Não havia mecanismo sistemático para saber se a tela entregue funciona para o usuário: onde ele trava, o que ignora, que fluxo abandona. Sem telemetria, a priorização de UX dependia de feedback anedótico — e melhorias eram reativas, não contínuas.

O risco central continua o mesmo. No instante em que se regenera em vez de distribuir, bifurca-se a fonte da verdade. O valor de um design system é "*uma definição, muitos consumidores*". Tudo nesta

arquitetura — inclusive as camadas novas de documentação e telemetria — existe para preservar essa fonte única: Supernova e Figma **consomem** os tokens do código; nunca os definem.

A v2 fecha essas duas pontas com o mesmo princípio da v1: automatizar o que é mecânico para que o time gaste energia apenas no que exige julgamento. Documentar vira pipeline; medir vira instrumentação; e o julgamento de UX passa a operar sobre dados.

O Ciclo Ponta a Ponta

O processo deixa de ser uma linha (descobrir → desenhar → entregar) e vira um **circuito de cinco estágios**. Cada estágio tem um responsável claro, ferramentas definidas e um critério de saída.



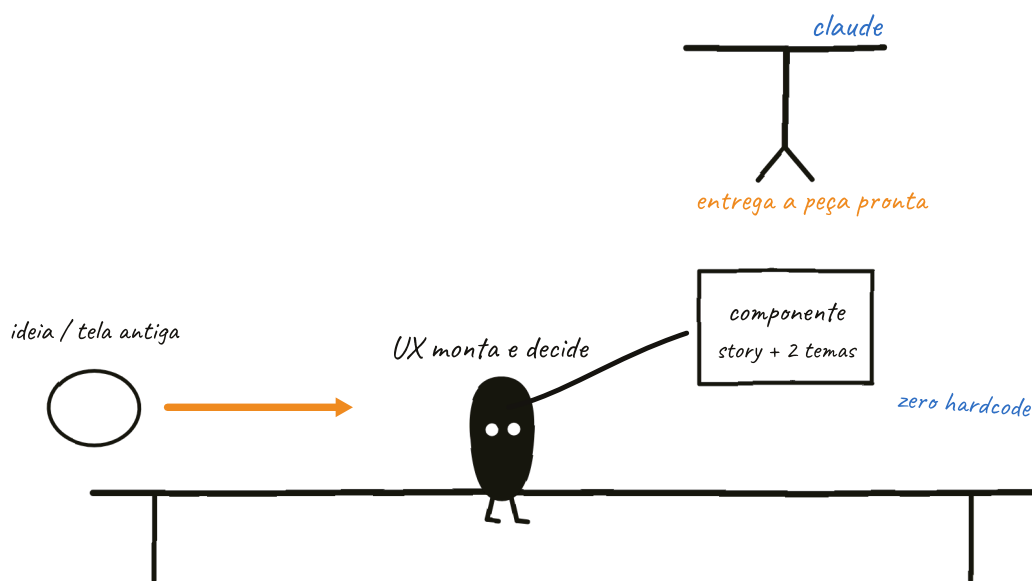
Descobrir → construir → entregar → documentar → medir — e o achado da medição reabastece o descobrir.

#	Estágio	O que acontece	Ferramentas
1	Descobrir	Insumos do ciclo: tela antiga como especificação (migração), dados do PostHog (replays, funis, surveys) e demandas de produto. UX define hipótese e escopo.	PostHog · produto legado
2	Construir	UX + Claude compõem a solução sobre o design system: tokens, primitivas do <code>@gb/ui</code> , receitas de composição. Zero hardcode, dois temas, todos os estados.	Claude + skill · <code>@gb/ui</code> · <code>@gb/tokens</code>
3	Entregar	CI valida (<code>@gb/lint</code> , type-check, stories). Commit convencional, revisão, merge. Componentes novos viram candidatos a promoção ao núcleo.	CI · Storybook · pipeline de promoção
4	Documentar	Depois da entrega: tokens fluem do código para o Figma (Tokens Studio) e para o Supernova; páginas de documentação montadas sobre dados vivos, não screenshots.	Supernova · Tokens Studio · Figma
5	Medir	PostHog captura uso real. Ritual mensal de leitura (replays, funis, rage clicks, surveys) gera achados priorizados que reabastecem o estágio 1.	PostHog

Definition of Done ampliado. Uma entrega só está completa quando: (a) passou no lint de tokens; (b) tem story no Storybook; (c) os eventos de uso relevantes estão instrumentados; (d) a documentação foi atualizada no ciclo de release. Documentação e instrumentação deixam de ser "depois, se der tempo".

Claude como Copiloto do Time de UX

Na v1, a IA aparecia principalmente do lado dos desenvolvedores — o Claude consumindo o catálogo e as receitas para montar telas. Na v2, o movimento se completa: o próprio time de UX opera com Claude em toda a cadeia de produção do design system. O UX deixa de "especificar para alguém construir" e passa a construir diretamente, com a IA executando o mecânico e o UX exercendo o julgamento.



A bancada: o UX monta e decide; o Claude entrega as peças já no padrão das Leis.

Frente	Como funciona
Tokens	Evoluir <code>tokens.ts</code> , gerar o <code>tokens.json</code> DTCG, propagar mudanças e preparar o changelog do bump de versão do <code>@gb/tokens</code> .
Componentes	Criar primitivas novas em <code>src/components/ui/</code> já com: tokens em todos os valores, suporte aos dois temas, story do Storybook, acessibilidade e todos os estados (loading, vazio, erro, denso, esparso).
Telas	Compor listagens e CRUDs inteiros seguindo as receitas canônicas (Regras B e C), importando apenas o catálogo — o equivalente funcional do mockup, direto em código.
Auditoria (AI-linting)	Varrer PRs e o código existente contra as Leis: <code>hardcode</code> , primitiva reinventada, tema faltando, estado não implementado. Sinaliza antes da revisão humana.
Documentação	Gerar o conteúdo das páginas do Supernova (uso, anatomia, do/don't de cada componente) a partir do código e das stories — o UX revisa, não redige do zero.

Frente	Como funciona
Promoção	Normalizar componentes criados pelos devs (tokenizar, completar estados) antes da publicação no <code>@gb/ui</code> — a engenharia reversa da federação, automatizada.

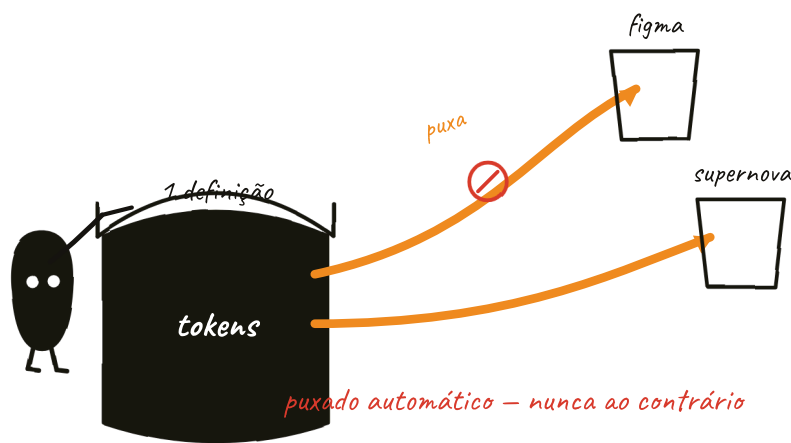
O contrato que torna isso confiável

O que impede a IA de gerar inconsistência é exatamente o investimento da v1: o `CLAUDE.md` com as Leis e Regras de composição, os tokens como vocabulário fechado e o catálogo como único caminho de construção. O Claude opera dentro dessa moldura — e o `@gb/lint` no CI verifica o resultado independentemente de quem (ou o quê) escreveu o código. Apenas **32%** dos profissionais confiam em output de IA sem contexto de design system; com a moldura, o output nasce conforme.

Papel humano não negociável. O Claude executa; o UX decide. Hierarquia de informação, fluxo, terminologia, trade-offs de densidade, decisões de produto — nada disso é delegável. A revisão de UX continua sendo o portão de qualidade de toda promoção ao núcleo e de toda tela nova. A IA remove o custo de produção, não o julgamento.

Documentação Viva — Supernova + Figma

A regra de ouro da documentação na v2: **documenta-se depois da entrega, puxando do código**. Nenhuma página de documentação contém valores digitados à mão — tokens, variáveis e exemplos vêm de fontes vivas (repositório, Figma variables, Storybook). Se o token muda no código, a documentação reflete a mudança no próximo sync, sem reescrita.



Um poço, muitos canos: Figma e Supernova bebem dos tokens do código — nunca o contrário.

#	Passo	Ferramenta
1	<code>tokens.ts</code> gera <code>tokens.json</code> no padrão W3C DTG (<code>\$value</code> / <code>\$type</code>) — automatizado no build, com ajuda do Claude.	Style Dictionary / script
2	Tokens Studio sincroniza o <code>tokens.json</code> do Git com as Figma Variables — o Figma passa a falar a língua dos tokens sem digitação manual.	Tokens Studio (plugin)
3	Supernova importa as variáveis do Figma (recurso do plano free) e o Storybook do CERNE como data sources.	Supernova
4	Páginas de documentação montadas no editor do Supernova referenciam os tokens e componentes importados — dados vivos, não screenshots.	Supernova

O que o plano Free do Supernova cobre	Limite Free
Usuários (full seats)	5
Design systems / versões	1 / 1

O que o plano Free do Supernova cobre	Limite Free
Data sources Figma / Storybook	3 / 1
Páginas de documentação	20
Pipelines de automação	1
Site de documentação	Privado
Import de Figma Variables	Incluído
Sync automático de data sources	Não (Pro)

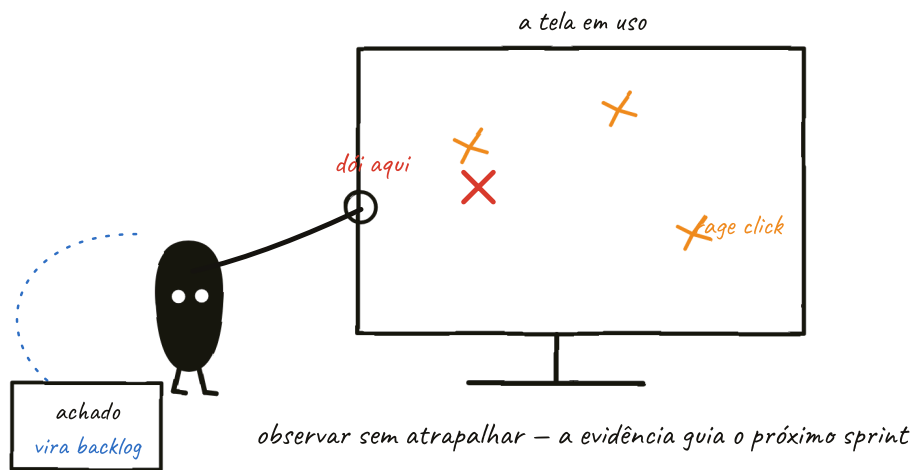
Limite mais relevante do free. A atualização automática das fontes de dados é recurso Pro. No primeiro momento isso não fere o processo: o sync manual entra como passo obrigatório do checklist de release (junto do bump do [@gb/tokens](#)). Se a frequência de releases tornar esse passo um gargalo — ou se 20 páginas deixarem de bastar — esses são os dois gatilhos objetivos para avaliar o upgrade (Seção 9).

O novo papel do Figma

O Figma sai de "entregável obrigatório" para *vitrine e laboratório*: recebe as variáveis automaticamente via Tokens Studio, serve de superfície para exploração visual quando útil (conceitos, marketing, onboarding de designers) e alimenta o Supernova como data source. O que não muda: o Figma nunca é onde um token nasce ou é editado — mudança de token nasce no código, com bump de versão e changelog. Manter o Figma *downstream* é o que o mantém barato.

Melhoria Contínua — PostHog

O PostHog fecha o circuito: instrumenta o CERNE e devolve comportamento real de uso — onde o produtor trava no cadastro de fazenda, que filtros ninguém usa, onde acontecem cliques de raiva. A priorização de UX passa a operar sobre evidência, e cada melhoria entregue é verificável na volta seguinte do ciclo.



Observar sem atrapalhar: replays e funis mostram onde dói; o achado vira backlog.

Produto	Limite Free (por mês)	Uso no processo
Product Analytics	1 milhão de eventos	Funis (cadastros multi-step), retenção, paths entre telas, web analytics.
Session Replay	5.000 gravações	Ver o uso real: hesitação, rage clicks, campos refeitos, abandono. Retenção de 1 mês no free.
Surveys	1.500 respostas	Micro-pesquisas contextuais (ex.: CSAT pós-cadastro de safra).
Feature Flags / Experiments	1 milhão de requests	Rollout gradual de telas migradas e testes A/B de variações de fluxo.
Error Tracking	100 mil exceções	Erros de UI correlacionados com replays.

O ritual de leitura

- **Mensal — Revisão de UX baseada em dados:** 60–90 min; o time lê os funis das jornadas críticas, assiste a uma amostra de replays das telas novas e revisa surveys. Saída: 3–5 achados priorizados que entram no estágio Descobrir do próximo ciclo.
- **Por release — Verificação de hipótese:** toda melhoria de UX relevante declara, antes de sair, qual métrica deve mover (ex.: "reduzir abandono no passo 2 do cadastro de fazenda"). Na revisão seguinte, confere-se. Sem isso, melhoria contínua vira opinião contínua.
- **Contínuo — Alertas:** picos de erro e quedas bruscas de conversão em fluxos críticos geram notificação, não esperam o ritual.

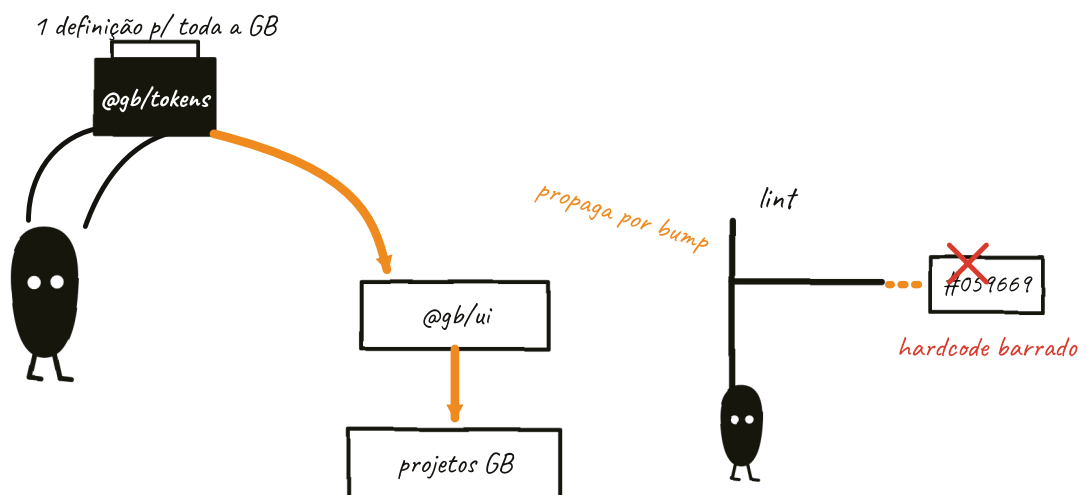
Métricas de produto que passam a existir

- Taxa de conclusão e tempo por etapa dos cadastros multi-step (Fazenda, Safra).
- Rage clicks e dead clicks por tela (atrito de interface).
- Adoção de telas migradas vs. produto legado (acompanha a migração).
- Uso real de filtros, busca e ações em massa nas listagens (informa o que promover ou simplificar no design system).

Privacidade e LGPD — pré-requisito, não detalhe. Session replay grava interação de usuários reais. Antes de ativar: mascaramento de inputs habilitado por padrão (dados pessoais, documentos, e-mails nunca gravados), captura restrita ao necessário, aviso de privacidade atualizado e validado com o jurídico, e acesso ao PostHog restrito ao time (princípio do menor privilégio). Anonimização e `person_profiles: 'identified_only'` na configuração inicial.

Arquitetura-Alvo

A base-alvo do design system proprietário GB é o **Chakra UI 3**: o tema passa a ser um `SystemConfig` alimentado pelos tokens (via `defineTokens` / `defineSemanticTokens`) — um esqueleto já existe em `src/theme.ts` (`createSystem`) — e os componentes do núcleo passam a ser construídos com recipes e slot recipes. A distribuição é por pacote npm versionado — o modelo que melhor serve "uma definição, muitos consumidores": uma correção no núcleo propaga para todos os projetos por bump de versão, sem cópias divergentes.



A constituição que propaga: o pote de tokens escorre para o núcleo e os projetos; o lint barra o hardcode.

```
@gb/tokens (npm | semver | TRAVADO)      ← a constituição: 1 definição p/ toda a GB
| tokens.json (DTCG) → script gera: Chakra tokens/semanticTokens ·
| Tailwind preset (transição CERNE) · CSS vars
▼
@gb/ui (npm | Chakra UI 3)                ← núcleo curado: SystemConfig
| mergeConfigs(defaultConfig, gbConfig) + recipes / slot recipes
▼
Projetos GB (CERNE, futuros)              ← createSystem(gbConfig); devs criam livremente
| enforcement: @gb/lint + strictTokens + typegen --strict no CI
| Claude: skill gb-design-system + MCP oficial do Chakra
▼
Pipeline de promoção (dev cria → UX aprova → normaliza → publica no @gb/ui)
▼
Documentação viva: tokens.json → Tokens Studio → Figma Variables → Supernova
▼
PostHog (uso real) → realimenta o backlog de UX
```

Camada	Formato	Modelo	Quem controla
Tokens	<code>@gb/tokens</code> (npm, semver)	Dependência travada; atualização central propaga	UX — estrito
Núcleo de componentes	<code>@gb/ui</code> (npm, Chakra UI 3)	Pacote versionado; recipes tematizáveis, extensão por props	UX cura; devs consomem
Componentes de projeto	Código no repo do dev	Federado, livre — sobre as primitivas do Chakra + tema GB	Devs criam; UX promove
Enforcement	<code>@gb/lint</code> + <code>strictTokens</code>	Obrigatório em todo repo GB (CI)	UX define a regra
Receitas de tela	Skill <code>gb-design-system</code>	Claude carrega sob demanda	UX mantém
Documentação	Supernova + Figma Variables	Downstream do código; sync por release	UX cura
Telemetria	PostHog (<code>posthog-js</code>)	Instrumentação contínua; leitura ritualizada	UX + produto

Por que Chakra UI 3 favorece o processo

- **strictTokens nativo** — com `strictTokens: true` no SystemConfig, o TypeScript só aceita valores de token nas props de estilo. A Lei 3 (tokenização) ganha enforcement em tempo de compilação, antes mesmo do lint.
- **chakra typegen** — gera tipos dos tokens e recipes do tema GB: autocomplete do vocabulário do design system na IDE de todo dev (e do Claude).
- **Semantic tokens e temas** — `defineSemanticTokens` com condições (`base` / `_dark`) resolve light + GBMode no nível do tema, eliminando lógica de tema espalhada por componente.
- **MCP oficial** — o `@chakra-ui/react-mcp` dá ao Claude acesso estruturado a props, exemplos e tema dos componentes Chakra; somado à skill GB, cobre descoberta sem custo de manutenção nosso.
- **Recipes como contrato** — variantes e tamanhos declarados em recipe são o contrato visual do componente; estender é adicionar variante na recipe do `@gb/ui`, nunca patch local (Lei 2 preservada).

Lacuna conhecida do ecossistema. Não existe (jun/2026) conversor oficial publicado de W3C DTCG para o formato de tokens do Chakra v3. O mapeamento, porém, é direto (`$value / $type` → `{ value }`): um script utilitário de ~50 linhas, de propriedade da GB, versionado dentro do `@gb/tokens` e gerado/mantido com o Claude. Risco baixo e controlado.

Plano de Implementação

Oito frentes — as seis da v1 (revisadas para Chakra) e duas novas. Sequência recomendada: 1 → 2 → 3+4 em paralelo → 5 e 6 contínuos; a frente 8 (telemetria) pode começar já na primeira semana, e a 7 (documentação) entra após a primeira leva de componentes estabilizar. A frente 1 continua destravando todas as outras.

Frente 1 — `@gb/tokens` : a pedra angular

`tokens.ts` → `tokens.json` (DTCG) → script gera três saídas: tokens/semanticTokens do Chakra (`defineTokens` / `defineSemanticTokens`), Tailwind preset (transição do CERNE atual) e CSS vars. Publicado como pacote npm versionado; todo projeto GB consome a mesma definição por construção.

Frente 2 — `@gb/lint` + `strictTokens`: o controle da federação

A Lei de tokenização vira tripla barreira: `strictTokens` no tema (compilação), `chakra typegen --strict` (tipos) e ESLint/Stylelint no CI (valores hardcoded). Criar componente continua livre — mas o que se cria é forçado a falar a língua dos tokens.

Frente 3 — `@gb/ui` : o núcleo curado sobre Chakra

Pacote npm com o SystemConfig GB (`mergeConfigs(defaultConfig, gbConfig)`) e as primitivas curadas reconstruídas sobre Chakra v3 com recipes/slot recipes. Semear com as mais reusadas do catálogo CERNE: Button, FormField, FormSelect, Card, Badge, DataTable, Modal, PageCard. Os snippets do Chakra CLI servem de ponto de partida de composição quando útil.

Frente 4 — Skill `gb-design-system` + MCP: o que substitui o Figma na construção

Empacotar Leis, Regras de composição (listagem e CRUD), contratos de componente e referência de tokens. O Claude de qualquer dev (e do UX) carrega a skill e monta telas novas sem Figma. O MCP oficial do Chakra complementa com props e exemplos das primitivas base. Versionada junto com os tokens.

Frente 5 — Pipeline de promoção: a engenharia reversa formalizada

- Dev cria o componente no projeto, sobre o tema GB.
- UX revisa contra critérios fixos: tokenizado, dois temas (semantic tokens), story, acessibilidade, todos os estados.
- Normalização: o Claude audita e propõe a versão em recipe/slot recipe — automatizável.
- Publicação no `@gb/ui` com bump semver: vira *commons* compartilhado.

Frente 6 — A migração

A tela antiga é a especificação (conteúdo e comportamento). Dev + Claude + skill + `@gb/ui` reconstroem cada tela; telas pós-migração apoiam-se 100% nas receitas. A migração alimenta o núcleo: componentes novos viram candidatos a promoção.

Frente 7 — Documentação viva **NOVO**

- Criar o workspace Supernova (free) e conectar data sources: arquivo Figma de variables + Storybook do CERNE.
- Configurar Tokens Studio sincronizando o `tokens.json` do Git com as Figma Variables.
- Curar as 20 páginas: visão geral, fundações (cor, tipo, espaço), 1 página por família de componentes, receitas de composição, governança.
- Claude gera o primeiro rascunho do conteúdo a partir do código e das stories; UX revisa e publica.
- Sync manual dos data sources entra no checklist de release (limite do plano free — 2 min por release).

Frente 8 — Telemetria e melhoria contínua **NOVO**

- Semana 1: `posthog-js` no `main.tsx` com mascaramento de inputs e `person_profiles: 'identified_only'`; validação de privacidade com o jurídico.
- Instrumentar as jornadas críticas: cadastros multi-step, listagens, busca e filtros.
- Ativar session replay e error tracking; definir os 3 primeiros funis.
- Agendar o ritual mensal de revisão de UX baseada em dados (Seção 6).

Governança e Modelo de Contribuição

O modelo permanece federado — tokens centralizados e estritos; componentes flexíveis na borda, com caminho de promoção ao núcleo — e ganha dois domínios novos: documentação e dados de uso. A governança continua sendo produto versionado, não documento.

Controles obrigatórios

Controle	O que faz	Modo de falha que previne
Token-lint + strictTokens no CI	Bloqueia valores hardcoded fora dos tokens — em compilação, tipos e lint	Drift visual entre produtos
Propriedade explícita	UX é dono dos tokens e do <code>@gb/ui</code> ; devs donos dos componentes de projeto	"Quem percebeu" conserta
Versionamento (semver)	Mudança de token ou de núcleo = bump = changelog = propaga	Atualizações silenciosas e quebras
Pipeline de promoção	Critério fixo para um componente virar núcleo	Sprawl de componentes redundantes
Documentação no DoD	Release só fecha com sync do Supernova/Figma executado	Documentação-espelho desatualizada
Privacidade por padrão	Mascaramento de replay, menor privilégio de acesso, conformidade LGPD revisada	Exposição de dados pessoais de usuários
Decisão baseada em dados	Melhoria de UX declara métrica-alvo antes; ritual mensal confere	Melhoria contínua virar opinião contínua
AI-linting (evoluir p/)	Agente audita PRs contra as Leis antes da revisão humana	Revisão manual cara e tardia

Gatilhos objetivos de upgrade (sair do free quando — e só quando)

Ferramenta	Gatilho	O que o pago destrava
Supernova Pro	Sync manual virou gargalo (releases frequentes) ou 20 páginas insuficientes ou necessidade de site público / viewers	Sync automático de data sources, mais páginas e pipelines, viewer seats
PostHog pago	Estouro consistente de 1M eventos/mês ou 5 mil replays — sinal de adoção, não de custo	Cobrança por uso acima do free; retenção maior de replays

Reposicionamento de discurso. Internamente na GB, vender isto não como "tooling de UX", mas como a pré-condição para que a IA gere código confiável em escala — e, com a v2, como o mecanismo que prova valor com dados: cada ciclo entrega evidência de melhoria, não percepção.

Riscos, Controles e Métricas de Saúde

Risco	Impacto	Controle
Bifurcação da fonte da verdade	N design systems divergentes	Tokens centrais + semver + lint + strictTokens
Sprawl de componentes	Redundância e inconsistência	Pipeline de promoção + "consultar o @gb/ui antes de criar"
Drift de tokens	Inconsistência visual entre produtos	@gb/lint obrigatório no CI
Documentação desatualizada	Supernova/Figma viram espelho morto (problema da v1 de volta)	Sync no checklist de release + documentação no DoD + dados vivos (nunca screenshots)
Dependência do operador (bus-factor)	Conhecimento na cabeça do UX	Leis e processo documentados + Storybook + Supernova publicado
Limites dos planos free	Gargalo de sync, páginas ou eventos	Gatilhos objetivos de upgrade (Seção 9); curadoria de páginas; amostragem de replays
Privacidade (LGPD) na telemetria	Exposição de dados pessoais; risco legal	Mascaramento por padrão, anonimização, acesso restrito, revisão jurídica antes de ativar
Lock-in de ferramenta de docs	Migração custosa se o Supernova mudar	Conteúdo curto e curado; tokens em DTCG (portável a 10+ ferramentas); export offline
Lacuna DTCG → Chakra	Script próprio a manter	~50 linhas, versionado no @gb/tokens , coberto por teste
Over-engineering de IA/MCP	Custo de contexto e manutenção	Começar com CLI + skill; MCP do Chakra é mantido pelo ecossistema, não por nós

Métricas de saúde do sistema

- **Cobertura de tokens** — % de valores de design vindos de tokens, por repositório (tendência: 100%).
- **Tempo de promoção** — da criação de um componente à publicação no @gb/ui .
- **Violações de lint** barradas no CI por sprint (tendência decrescente).
- **Adoção** — nº de produtos GB consumindo @gb/tokens e @gb/ui .
- **Frescor da documentação** **NOVO** — releases com sync executado / total de releases (meta: 100%).

- **Saúde de uso** **NOVO** — conclusão dos fluxos críticos, rage clicks por tela, adoção das telas migradas (PostHog).
- **Ciclo fechado** **NOVO** — % de melhorias de UX entregues com hipótese declarada e verificada no ritual seguinte.

Glossário

Termo	Definição
Design token	Valor nomeado de design (cor, espaço, raio, tipografia) que substitui literais hardcoded.
W3C DTCG	Design Tokens Community Group; formato-padrão (JSON com <code>\$value / \$type</code>) estável desde out/2025.
SSOT	Single Source of Truth — a fonte única e autoritativa de uma definição.
Chakra UI v3	Biblioteca de componentes React com theming por tokens (<code>createSystem</code>); base do design system proprietário GB.
Recipe / slot recipe	Mecanismo do Chakra para declarar variantes e estilos de um componente (simples ou multi-parte) no tema.
Semantic token	Token que resolve para valores diferentes por condição (ex.: light vs. GBMode), definido no tema.
strictTokens	Opção do Chakra que faz o TypeScript aceitar apenas tokens nas props de estilo.
Style Dictionary	Ferramenta que transforma um arquivo de tokens em múltiplos formatos (CSS, Tailwind etc.); lê DTCG desde a v4.
Tokens Studio	Plugin Figma que sincroniza variáveis com um arquivo de tokens em Git.
Supernova	Plataforma que conecta código, Figma e Storybook para gerar documentação viva de design system.
PostHog	Plataforma open-source de product analytics: eventos, session replay, feature flags, experiments e surveys.
Session replay	Gravação anonimizada e mascarada da interação do usuário com a interface.
Feature flag	Chave de ativação remota que permite rollout gradual e testes A/B sem novo deploy.
Rage click	Cliques repetidos e rápidos no mesmo elemento — sinal clássico de atrito de interface.
MCP	Model Context Protocol — protocolo que dá a agentes de IA contexto e ferramentas estruturadas.
Federação	Modelo onde times criam componentes localmente, com promoção curada ao núcleo compartilhado.

Termo	Definição
Flywheel	Ciclo virtuoso: o design system melhora a IA, que reforça o design system — agora com dados de uso fechando a volta.

Referências

- W3C Design Tokens Community Group — especificação estável v2025.10 · design-tokens.github.io/community-group
- Chakra UI v3 — Theming (createSystem, tokens, semantic tokens, recipes) · chakra-ui.com/docs/theming/overview
- Chakra UI — CLI (typegen, snippet, eject) · chakra-ui.com/docs/get-started/cli
- Chakra UI — MCP Server oficial · npmjs.com/package/@chakra-ui/react-mcp
- Supernova — Pricing e recursos do plano Free · supernova.io/pricing
- Supernova — Documentação (data sources Figma e Storybook) · learn.supernova.io
- Tokens Studio for Figma — sync de tokens via Git · tokens.studio
- PostHog — Pricing (free tier) · posthog.com/pricing
- PostHog — biblioteca React/JS · posthog.com/docs/libraries/react
- Figma — Design Systems and AI: Why MCP Servers Are the Unlock · figma.com/blog/design-systems-ai-mcp
- Design Token SSOT: Figma or Code? — Design Systems Collective · designsystemscollective.com
- Figma is not the source of truth for design tokens — Artur Sopelnik · artursopelnik.de

Documento gerado a partir do desenho de processo validado com a equipe de UX da GB Agritech, da v1.0 (Processo de UX Code-First) e de verificação das capacidades das ferramentas citadas. Dados de planos gratuitos conferidos em junho/2026 — sujeitos a alteração pelos fornecedores. Ilustrações no estilo Xiaohei (traço à mão, fundo branco) desenhadas especialmente para esta edição.